Лекция №4.
OpenMP. Основы.



Григорьев Василий Васильевич

2023



Введение

- Одним из наиболее популярных средств программирования для компьютеров с общей памятью, базирующихся на традиционных языках программирования и использовании специальных комментариев, в настоящее время является технология OpenMP. За основу берётся последовательная программа, а для создания её параллельной версии пользователю предоставляется набор директив, функций и переменных окружения. Предполагается, что создаваемая параллельная программа будет переносимой между различными компьютерами с разделяемой памятью, поддерживающими OpenMP API.

Введение

- Технология OpenMP нацелена на то, чтобы пользователь имел один вариант программы для параллельного и последовательного выполнения. Однако возможно создавать программы, которые работают корректно только в параллельном режиме или дают в последовательном режиме другой результат. Более того, из-за накопления ошибок округления результат вычислений с использованием различного количества нитей может в некоторых случаях различаться.

Введение

- Разработкой стандарта занимается некоммерческая организация OpenMP ARB (Architecture Review Board), в которую вошли представители крупнейших компаний – разработчиков SMP-архитектур и программного обеспечения. OpenMP поддерживает работу с языками Фортран и Си/Си++. Первая спецификация для языка Фортран появилась в октябре 1997 года, а спецификация для языка Си/Си++ – в октябре 1998 года. На данный момент последняя официальная спецификация стандарта – OpenMP 5.2 (принята в ноябре 2021 года).

Введение

- Интерфейс OpenMP задуман как стандарт для программирования на масштабируемых SMP-системах (SSMP, ccNUMA и других) в модели общей памяти (shared memory model). В стандарт OpenMP входят спецификации набора директив компилятора, вспомогательных функций и переменных среды. OpenMP реализует параллельные вычисления с помощью многопоточности, в которой «главный» (master) поток создает набор «подчиненных» (slave) потоков, и задача распределяется между ними. Предполагается, что потоки выполняются параллельно на машине с несколькими процессорами, причём количество процессоров не обязательно должно быть больше или равно количеству потоков.

Как установить?

- Если вы до этого без проблем компилировали и запускали программы написанные на MPI, то дополнительно ничего устанавливать не надо.
- Важно наличие g++ компилятора. В прошлых презентациях была подробная инструкция.

Моя первая программа на OpenMP

```
#include <omp.h>
#include <iostream>
#define NUM_THREADS 4
using namespace std;

int main(int argc, char *argv[])
{
  int th_id, nthreads;
  omp_set_num_threads(NUM_THREADS);
  #pragma omp parallel private(th_id) shared(nthreads)
  {
    th_id = omp_get_thread_num();
    #pragma omp critical
    {
      cout << "Hello World from thread " << th_id << '\n';
    }
    #pragma omp barrier

    #pragma omp master
    {
      nthreads = omp_get_num_threads();
      cout << "There are " << nthreads << " threads" << '\n';
    }
  }

  return 0;
}
```

Как запустить?

g++ -Wall -fopenmp hello.cpp -o hello

./hello

```
vasiliy@BEAST:/mnt/d/Dropbox/lol/Вычислительные системы и
/Программы/OpenMP$ g++ -Wall -fopenmp hello.cpp -o hello
vasiliy@BEAST:/mnt/d/Dropbox/lol/Вычислительные системы и
/Программы/OpenMP$ ./hello
Hello World from thread 3
Hello World from thread 4
Hello World from thread 6
Hello World from thread 5
Hello World from thread 1
Hello World from thread 2
Hello World from thread 0
Hello World from thread 7
There are 8 threads
```

Модель параллельной программы

- Распараллеливание в OpenMP выполняется явно при помощи вставки в текст программы специальных директив, а также вызова вспомогательных функций;
- Программа начинается с последовательной области – сначала работает один процесс (нить), при входе в параллельную область порождается ещё некоторое число процессов, между которыми в дальнейшем распределяются части кода;
- Для написания эффективной параллельной программы необходимо, чтобы все нити, участвующие в обработке программы, были равномерно загружены полезной работой.

Модель параллельной программы

- Существенным моментом является также необходимость синхронизации доступа к общим данным. Само наличие данных, общих для нескольких нитей, приводит к конфликтам при одновременном несогласованном доступе. Поэтому значительная часть функциональности OpenMP предназначена для осуществления различного рода синхронизаций работающих нитей;
- OpenMP не выполняет синхронизацию доступа различных нитей к одним и тем же файлам. Если это необходимо для корректности программы, пользователь должен явно использовать директивы синхронизации или соответствующие библиотечные функции. При доступе каждой нити к своему файлу никакая синхронизация не требуется.

Директивы и функции

- Значительная часть функциональности OpenMP реализуется при помощи директив компилятору. Они должны быть явно вставлены пользователем, что позволит выполнять программу в параллельном режиме.
- Директивы OpenMP в программах на языке Си оформляются указаниями препроцессору, начинающимися с #pragma omp. Ключевое слово omp используется для того, чтобы исключить случайные совпадения имён директив OpenMP с другими именами.

Формат директивы на Си/Си++:

#pragma omp *directive-name* [опция[[,] опция]...]

Пояснение

- Объектом действия большинства директив является один оператор или блок, перед которым расположена директива в исходном тексте программы. В OpenMP такие операторы или блоки называются ассоциированными с директивой. Ассоциированный блок должен иметь одну точку входа в начале и одну точку выхода в конце. Порядок опций в описании директивы несущественен, в одной директиве большинство опций может встречаться несколько раз. После некоторых опций может следовать список переменных, разделяемых запятыми.
- Все директивы OpenMP можно разделить на 3 категории: определение параллельной области, распределение работы, синхронизация. Каждая директива может иметь несколько дополнительных атрибутов – опций (clause). Отдельно специфицируются опции для назначения классов переменных, которые могут быть атрибутами различных директив.

Пример с замером времени выполнения

```cpp
#include <iostream>
#include <omp.h>

using namespace std;
int main(int argc, char *argv[])
{
 double start_time, end_time, tick;
 start_time = omp_get_wtime();

 end_time = omp_get_wtime();
 tick = omp_get_wtick();
 cout<< "Время на замер времени:" << end_time-start_time<<endl;
 cout<<"Точность таймера: " << tick<<endl;
}
```

```
vasiliy@BEAST:/mnt/d/Dropbox/lol/Вы
Время на замер времени:1.55e-07
Точность таймера: 1e-09
```

Выполнение программы. Параллельные и последовательные области

- В момент запуска программы порождается единственная нить-мастер или «основная» нить, которая начинает выполнение программы с первого оператора. Основная нить и только она исполняет все последовательные области программы. При входе в параллельную область порождаются дополнительные нити.

- Параллельная область задаётся при помощи директивы parallel.

#pragma omp parallel [опция[[,] опция]...]

Возможные опции

- **if(условие)** – выполнение параллельной области по условию. Вхождение в параллельную область осуществляется только при выполнении некоторого условия. Если условие не выполнено, то директива не срабатывает и продолжается обработка программы в прежнем режиме;
- **num_threads (целочисленное выражение)** – явное задание количества нитей, которые будут выполнять параллельную область; по умолчанию выбирается последнее значение, установленное с помощью функции omp_set_num_threads(), или значение переменной OMP_NUM_THREADS;
- **default(private|firstprivate|shared|none)** – всем переменным в параллельной области, которым явно не назначен класс, будет назначен класс shared или none; none означает, что всем переменным в параллельной области класс должен быть назначен явно;
- **private(список)** – задаёт список переменных, для которых порождается локальная копия в каждой нити; начальное значение локальных копий переменных из списка не определено;

Возможные опции

- **firstprivate(список)** – задаёт список переменных, для которых порождается локальная копия в каждой нити; локальные копии переменных инициализируются значениями этих переменных в нити-мастере;
- **shared(список)** – задаёт список переменных, общих для всех нитей;
- **copyin(список)** – задаёт список переменных, объявленных как threadprivate, которые при входе в параллельную область инициализируются значениями соответствующих переменных в нити-мастере;
- **reduction(оператор:список)** – задаёт оператор и список общих переменных; для каждой переменной создаются локальные копии в каждой нити; локальные копии инициализируются соответственно типу оператора (для аддитивных операций – 0 или его аналоги, для мультипликативных операций – 1 или её аналоги); над локальными копиями переменных после выполнения всех операторов параллельной области выполняется заданный оператор; оператор это: – +, *, -, &, |, ^, &&, ||; порядок выполнения операторов не определён, поэтому результат может отличаться от запуска к запуску.

Нюанс

- При входе в параллельную область порождаются новые OMP_NUM_THREADS - 1 нитей, каждая нить получает свой уникальный номер, причём порождающая нить получает номер 0 и становится основной нитью группы («мастером»). Остальные нити получают в качестве номера целые числа с 1 до OMP_NUM_THREADS - 1.

- Количество нитей, выполняющих данную параллельную область, остаётся неизменным до момента выхода из области. При выходе из параллельной области производится неявная синхронизация и уничтожаются все нити, кроме породившей.

Пример. parallel

```
#include <iostream>
using namespace std;
int main(int argc, char *argv[])
{
    cout<<"Последовательная область 1\n";
    #pragma omp parallel
    {
    cout<<"Параллельная область\n";
    }
    cout<<"Последовательная область 2\n";
}
```

```
vasiliy@BEAST:/mnt/d/Dropbox/
Последовательная область 1
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Параллельная область
Последовательная область 2
```

Базовая скорость:	3,40 ГГц
Сокетов:	1
Ядра:	16
Логических процессоров:	24
Виртуализация:	Включено
Кэш L1:	1,4 МБ
Кэш L2:	24,0 МБ
Кэш L3:	30,0 МБ

Пример. reduction

```
#include <iostream>
using namespace std;
int main(int argc, char *argv[])
{
    int count = 0;
    #pragma omp parallel reduction (+: count)
    {
    count++;
    cout<<"Текущее значение count: " <<count<<endl;
    }
    cout<<"Число нитей: " << count<<endl;
    return 0;
}
```

В данном примере производится подсчет общего количества порождённых нитей. Каждая нить инициализирует локальную копию переменной count значением 0. Далее, каждая нить увеличивает значение собственной копии переменной count на единицу и выводит полученное число. На выходе из параллельной области происходит суммирование значений переменных count по всем нитям, и полученная величина становится новым значением переменной count в последовательной области.

```
Текущее значение count: 1
Текущее значение count: 1
Текущее значение count: 1
Текущее значение count: 1
Текущее значение count: Тек
Текущее значение count: 1
Текущее значение count: 1
Текущее значение count: 1
Текущее значение count: 1
Текущее значение count: 1
Текущее значение count: 1
Текущее значение count: 1
Текущее значение count: 1
Текущее значение count: Тек
Текущее значение count: 1
Текущее значение count: 1
1
Текущее значение count: 1
Текущее значение count: 1
Текущее значение count: Тек
1
1
1
Текущее значение count: 1
Текущее значение count: 1
Число нитей: 24
```

Пример задания количества нитей

```
#include <iostream>
#include <omp.h>
using namespace std;
int main(int argc, char *argv[])
{
    omp_set_num_threads(2);
    cout<<"Последовательная область 1\n";
    #pragma omp parallel
    {
    cout<<"Параллельная область\n";
    }
    cout<<"Последовательная область 2\n";
}
```

```
Последовательная область 1
Параллельная область
Параллельная область
Последовательная область 2
```

Динамическое изменение количества нитей

```
#include <iostream>
#include <omp.h>
using namespace std;

int main(int argc, char *argv[])
{
    cout << "Значение OMP_DYNAMIC: " << omp_get_dynamic() << endl;
    omp_set_dynamic(1);
    cout << "Значение OMP_DYNAMIC: " << omp_get_dynamic() << endl;
    #pragma omp parallel num_threads(128)
    {
        #pragma omp master
        {
            cout << "Параллельная область, " << omp_get_num_threads() << " нитей\n";
            ;
        }
    }
}
```

```
Значение OMP_DYNAMIC: 0
Значение OMP_DYNAMIC: 1
Параллельная область, 24 нитей
```

В некоторых случаях система может динамически изменять количество нитей, используемых для выполнения параллельной области, например, для оптимизации использования ресурсов системы. Это разрешено делать, если переменная среды OMP_DYNAMIC установлена в true.
В параллельной области печатается реальное число выполняющих её нитей. Директива master позволяет обеспечить печать только процессом-мастером. В системах с динамическим изменением числа нитей выданное значение может отличаться от заданного (128).

Функции

- Функция omp_get_max_threads() возвращает максимально допустимое число нитей для использования в следующей параллельной области.
- Функция omp_get_num_procs() возвращает количество процессоров, доступных для использования программе пользователя на момент вызова. Нужно учитывать, что количество доступных процессоров может динамически изменяться.
- Функция omp_set_nested() разрешает или запрещает вложенный параллелизм. На языке Си в качестве значения параметра задаётся 0 или 1. Если вложенный параллелизм разрешён, то каждая нить, в которой встретится описание параллельной области, породит для её выполнения новую группу нитей. Сама породившая нить станет в новой группе нитью-мастером. Если система не поддерживает вложенный параллелизм, данная функция не будет иметь эффекта.

Пример вложенных параллельных областей

```
#include <iostream>
#include <omp.h>
using namespace std;

int main(int argc, char *argv[])
{
    int n;
    omp_set_nested(1);
    #pragma omp parallel private(n)
    {
        n=omp_get_thread_num();
        #pragma omp parallel
        {
            cout<<"Часть 1, нить " << n << " - " <<
omp_get_thread_num() <<endl;
        }
    }
    omp_set_nested(0);
    #pragma omp parallel private(n)
    {
        n=omp_get_thread_num();
        #pragma omp parallel
        {
            cout<<"Часть 2, нить " << n << " - " <<
omp_get_thread_num() <<endl;
        }
    }
}
```

Пример демонстрирует использование вложенных параллельных областей и функции omp_set_nested(). Вызов функции omp_set_nested() перед первой частью разрешает использование вложенных параллельных областей. Для определения номера нити в текущей параллельной секции используются вызовы функции omp_get_thread_num(). Каждая нить внешней параллельной области породит новые нити, каждая из которых напечатает свой номер вместе с номером породившей нити. Далее вызов omp_set_nested() запрещает использование вложенных параллельных областей. Во второй части вложенная параллельная область будет выполняться без порождения новых нитей.

Функция omp_parallel()

```
#include <iostream>
#include <omp.h>
using namespace std;

void mode(void)
{
    if(omp_in_parallel())
cout<<"Параллельная область\n";
    else cout<<"Последовательная
область\n";
}
```

```
int main(int argc, char *argv[])
{
    mode();
    #pragma omp parallel
    {
        #pragma omp master
        {
            mode();
        }
    }
}
```

```
Последовательная область
Параллельная область
```

Директива single

- Если в параллельной области какой-либо участок кода должен быть выполнен лишь один раз, то его нужно выделить директивами single

#pragma omp single [опция [[,] опция]...]

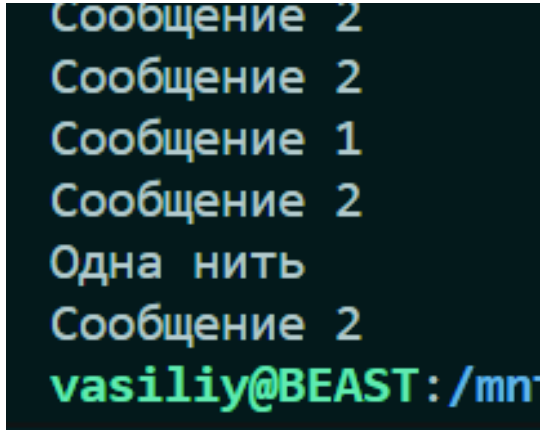
- Какая именно нить будет выполнять выделенный участок программы, не специфицируется. Одна нить будет выполнять данный фрагмент, а все остальные нити будут ожидать завершения её работы, если только не указана опция nowait. Необходимость использования директивы single часто возникает при работе с общими переменными.

Возможные опции

- **private(список)** – задаёт список переменных, для которых порождается локальная копия в каждой нити; начальное значение локальных копий переменных из списка не определено;
- **firstprivate(список)** – задаёт список переменных, для которых порождается локальная копия в каждой нити; локальные копии переменных инициализируются значениями этих переменных в нити-мастере;
- **copyprivate(список)** – после выполнения нити, содержащей конструкцию single, новые значения переменных списка будут доступны всем одноименным частным переменным (private и firstprivate), описанным в начале параллельной области и используемым всеми её нитями; опция не может использоваться совместно с опцией nowait; переменные списка не должны быть перечислены в опциях private и firstprivate данной директивы single;
- **nowait** – после выполнения выделенного участка происходит неявная барьерная синхронизация параллельно работающих нитей: их дальнейшее выполнение происходит только тогда, когда все они достигнут данной точки; если в подобной задержке нет необходимости, опция nowait позволяет нитям, уже дошедшим до конца участка, продолжить выполнение без синхронизации с остальными.

Пример применения

```
#include <iostream>
using namespace std;
int main(int argc, char *argv[])
{
    #pragma omp parallel
    {
        cout<<"Сообщение 1\n";
        #pragma omp single nowait
        {
            cout<<"Одна нить\n";
        }
        cout<<"Сообщение 2\n";
    }
}
```



Если убрать опцию nowait, то по окончании области single произойдёт барьерная синхронизация, и ни одна выдача "Сообщение 2" не может появиться до выдачи "Одна нить".

Пример copyprivate

```c
#include <stdio.h>
#include <omp.h>
int main(int argc, char *argv[])
{
 int n;
#pragma omp parallel private(n)
 {
 n=omp_get_thread_num();
 printf("Значение n (начало): %d\n", n);
#pragma omp single copyprivate(n)
 {
 n=100;
 }
 printf("Значение n (конец): %d\n", n);
 }
}
```

```
vasiliy@BEAST:/mnt/d/Drop
Значение n (начало): 0
Значение n (начало): 11
Значение n (начало): 5
Значение n (начало): 21
Значение n (начало): 14
Значение n (начало): 9
Значение n (начало): 16
Значение n (начало): 10
Значение n (начало): 15
Значение n (начало): 22
Значение n (начало): 4
Значение n (начало): 7
Значение n (начало): 12
Значение n (начало): 23
Значение n (начало): 2
Значение n (начало): 17
Значение n (начало): 6
Значение n (начало): 8
Значение n (начало): 20
Значение n (начало): 13
Значение n (начало): 3
Значение n (начало): 19
Значение n (начало): 1
Значение n (начало): 18
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
Значение n (конец): 100
vasiliy@BEAST:/mnt/d/Drop
```

В данном примере переменная n объявлена в параллельной области как локальная. Каждая нить присвоит переменной n значение, равное своему порядковому номеру, и напечатает данное значение. В области single одна из нитей присвоит переменной n значение 100, и на выходе из области это значение будет присвоено переменной n на всех нитях. В конце параллельной области значение n печатается ещё раз и на всех нитях оно равно 100.

И только тут я понял что…

- При работе лучше использовать Сишный printf вместо cout.
  Из-за специфики выполнения потокового вывода, в параллельной программе возможен вывод в «одно мясо».

```
Значение n (начало): Значение n (начало): 18
Значение n (начало): Значение n (начало): 212
Значение n (начало): Значение n (начало): 5
Значение n (начало): 9
23Значение n (начало): 3
```

```
Часть 2, нить Часть 2, нить Часть 2, нить Часть 2, ни
асть 2, нить Часть 2, нить 149818 - 016
Часть 2, нить 10 - 0
7Часть 2, нить 17 - 0
Часть 2, нить 23 - 0
 - Часть 2, нить 0
134 - 0
Часть 2, нить 3 - 0
 - Часть 2, нить 5120 - 0
Часть 2, нить Часть 2, нить Часть 2, нить 2122 - 0
 - 2Часть 2, нить Часть 2, нить 1912 - 0
Часть 2, нить  -  - 060
```

```
Текущее значение count: Текущее значение count: Текущее значение count: 1
```

Директива master

```
#include <stdio.h>
int main(int argc, char *argv[])
{
    int n;
    #pragma omp parallel private(n)
    {
        n=1;
        #pragma omp master
        {
            n=2;
        }
        printf("Первое значение n: %d\n",
n);
        #pragma omp barrier
        #pragma omp master
        {
            n=3;
        }
        printf("Второе значение n: %d\n",
n);
    }
}
```

```
vasiliy@BEAST:/mnt/d/D
Первое значение n: 1
Первое значение n: 1
Первое значение n: 1
Первое значение n: 1
Первое значение n: 1
Первое значение n: 1
Первое значение n: 1
Первое значение n: 2
Первое значение n: 1
Первое значение n: 1
Первое значение n: 1
Первое значение n: 1
Первое значение n: 1
Первое значение n: 1
Первое значение n: 1
Первое значение n: 1
Первое значение n: 1
Первое значение n: 1
Первое значение n: 1
Первое значение n: 1
Первое значение n: 1
Второе значение n: 1
Второе значение n: 1
Второе
```

Слегка модернизируем примерчик

```
#include <stdio.h>
#include <omp.h>
int main(int argc, char *argv[])
{
    int n, th_id;
    #pragma omp parallel private(n, th_id)
    {
        th_id = omp_get_thread_num();
        n=1;
        #pragma omp master
        {
            n=2;
        }
        printf("Первое значение n: %d и номер процесса: %d\n", n, th_id);
        #pragma omp barrier
        #pragma omp master
        {
            n=3;
        }
        printf("Второе значение n: %d и номер процесса: %d\n", n, th_id);
    }
}
```

```
vasiliy@BEAST:/mnt/d/Dropbox/lol/Вычислите
Первое значение n: 1 и номер процесса: 6
Первое значение n: 1 и номер процесса: 8
Первое значение n: 1 и номер процесса: 17
Первое значение n: 1 и номер процесса: 14
Первое значение n: 1 и номер процесса: 9
Первое значение n: 1 и номер процесса: 4
Первое значение n: 1 и номер процесса: 18
Первое значение n: 1 и номер процесса: 13
Первое значение n: 1 и номер процесса: 19
Первое значение n: 2 и номер процесса: 0
Первое значение n: 1 и номер процесса: 20
Первое значение n: 1 и номер процесса: 23
Первое значение n: 1 и номер процесса: 15
Первое значение n: 1 и номер процесса: 22
Первое значение n: 1 и номер процесса: 11
Первое значение n: 1 и номер процесса: 10
Первое значение n: 1 и номер процесса: 12
Первое значение n: 1 и номер процесса: 7
Первое значение n: 1 и номер процесса: 1
Первое значение n: 1 и номер процесса: 21
Первое значение n: 1 и номер процесса: 5
Первое значение n: 1 и номер процесса: 2
Первое значение n: 1 и номер процесса: 16
Первое значение n: 1 и номер процесса: 3
Второе значение n: 1 и номер процесса: 20
Второе значение n: 1 и номер процесса: 5
Второе значение n: 1 и номер процесса: 17
Второе значение n: 1 и номер процесса: 9
Второе значение n: 3 и номер процесса: 0
Второе значение n: 1 и номер процесса: 16
Второе значение n: 1 и номер процесса: 1
Второе значение n: 1 и номер процесса: 6
Второе значение n: 1 и номер процесса: 8
Второе значение n: 1 и номер процесса: 7
Второе значение n: 1 и номер процесса: 3
Второе значение n: 1 и номер процесса: 4
Второе значение n: 1 и номер процесса: 22
Второе значение n: 1 и номер процесса: 14
Второе значение n: 1 и номер процесса: 19
Второе значение n: 1 и номер процесса: 23
Второе значение n: 1 и номер процесса: 21
Второе значение n: 1 и номер процесса: 13
Второе значение n: 1 и номер процесса: 15
Второе значение n: 1 и номер процесса: 18
Второе значение n: 1 и номер процесса: 12
Второе значение n: 1 и номер процесса: 11
Второе значение n: 1 и номер процесса: 10
Второе значение n: 1 и номер процесса: 2
vasiliy@BEAST:/mnt/d/Dropbox/lol/Вычислите
```

Модель данных

Модель данных в OpenMP предполагает наличие как общей для всех нитей области памяти, так и локальной области памяти для каждой нити. В OpenMP переменные в параллельных областях программы разделяются на два основных класса:

- shared (общие; все нити видят одну и ту же переменную);
- private (локальные, приватные; каждая нить видит свой экземпляр данной переменной).

Опасность

Если несколько переменных одновременно записывают значение общей переменной без выполнения синхронизации или если как минимум одна нить читает значение общей переменной и как минимум одна нить записывает значение этой переменной без выполнения синхронизации, то возникает ситуация так называемой «гонки данных» (data race), при которой результат выполнения программы непредсказуем.

Нюансы

По умолчанию, все переменные, порождённые вне параллельной области, при входе в эту область остаются общими (shared). Исключение составляют переменные, являющиеся счетчиками итераций в цикле, по очевидным причинам. Переменные, порождённые внутри параллельной области, по умолчанию являются локальными (private). Явно назначить класс переменных по умолчанию можно с помощью опции default. Не рекомендуется постоянно полагаться на правила по умолчанию, для большей надёжности лучше всегда явно описывать классы используемых переменных, указывая в директивах OpenMP опции private, shared, firstprivate, lastprivate, reduction.

Пример private

```
#include <stdio.h>
#include <omp.h>
int main(int argc, char *argv[])
{
omp_set_num_threads(8);
 int n=1;
 printf("n в последовательной области (начало): %d\n", n);
#pragma omp parallel private(n)
 {
 printf("Значение n на нити (на входе): %d\n", n);
 /* Присвоим переменной n номер текущей нити */
 n=omp_get_thread_num();
 printf("Значение n на нити (на выходе): %d\n", n);
 }
 printf("n в последовательной области (конец): %d\n", n);
}
```

```
vasiliy@BEAST:/mnt/d/Dropbox/lol/Вычислит
n в последовательной области (начало): 1
Значение n на нити (на входе): 32630
Значение n на нити (на выходе): 0
Значение n на нити (на входе): 0
Значение n на нити (на выходе): 7
Значение n на нити (на входе): 0
Значение n на нити (на выходе): 6
Значение n на нити (на входе): 0
Значение n на нити (на выходе): 2
Значение n на нити (на входе): 0
Значение n на нити (на выходе): 5
Значение n на нити (на входе): 0
Значение n на нити (на выходе): 1
Значение n на нити (на входе): 0
Значение n на нити (на выходе): 3
Значение n на нити (на входе): 0
Значение n на нити (на выходе): 4
n в последовательной области (конец): 1
vasiliy@BEAST:/mnt/d/Dropbox/lol/Вычислит
```

Пример shared

```
#include <stdio.h>
#include <omp.h>
int main(int argc, char *argv[])
{
    int i, m[10];
    printf("Массив m в начале:\n");
    /* Заполним массив m нулями и напечатаем его */
    for (i=0; i<10; i++)
    {
        m[i]=0;
        printf("%d\n", m[i]);
    }
    #pragma omp parallel shared(m)
    {
        /* Присвоим 1 элементу массива m, номер которого
        совпадает с номером текущий нити */
        m[omp_get_thread_num()]=omp_get_thread_num();
    }
    /* Ещё раз напечатаем массив */
    printf("Массив m в конце:\n");
    for (i=0; i<10; i++)
        printf("%d\n", m[i]);
}
```

```
vasiliy@BEAST:/mnt/
Массив m в начале:
0
0
0
0
0
0
0
0
0
0
Массив m в конце:
0
1
2
3
4
5
6
7
8
9
*** stack smashing
Aborted
```

Пример firstprivate

```c
#include <stdio.h>
#include <omp.h>
int main(int argc, char *argv[])
{
    omp_set_num_threads(8);
    int n=1;
    printf("Значение n в начале: %d\n", n);
    #pragma omp parallel firstprivate(n)
    {
    printf("Значение n на нити (на входе): %d\n", n);
    /* Присвоим переменной n номер текущей нити */
    n=omp_get_thread_num();
    printf("Значение n на нити (на выходе): %d\n", n);
    }
    printf("Значение n в конце: %d\n", n);
}
```

```
vasiliy@BEAST:/mnt/d/Dropbox/lol/Вы
Значение n в начале: 1
Значение n на нити (на входе): 1
Значение n на нити (на выходе): 0
Значение n на нити (на входе): 1
Значение n на нити (на выходе): 2
Значение n на нити (на входе): 1
Значение n на нити (на выходе): 5
Значение n на нити (на входе): 1
Значение n на нити (на выходе): 4
Значение n на нити (на входе): 1
Значение n на нити (на выходе): 6
Значение n на нити (на входе): 1
Значение n на нити (на выходе): 1
Значение n на нити (на входе): 1
Значение n на нити (на выходе): 3
Значение n на нити (на входе): 1
Значение n на нити (на выходе): 7
Значение n в конце: 1
vasiliy@BEAST:/mnt/d/Dropbox/lol/Вы
```

Хватит на сегодня. Спасибо за внимание!

Разработчик: Это понятная фича, документация не нужна.
Пользователь:

